

Отчёт по лабораторной работе №24

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Миронова Татьяна Михайловна № по списку 12.

Контакты: tatirona@gmail.com

Работа выполнена: «27» мая 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «27» мая 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** AST

2. **Цель работы:** Составить программу на языке Си с выполнением арифметических выражений с помощью деревьев.

3. **Задание:** 12. Вывести общие сомножители (переменные и константы) из разности

4. **Оборудование:** *Оборудование ПЭВМ студента, если использовалось:*

Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ

5. **Программное обеспечение:** *Программное обеспечение ЭВМ студента, если использовалось:*

Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд bash версия 5.2.21.

Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.

Язык программирования: C.

6. **Идея, метод, алгоритм решения задачи** [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:

В основе решения лежит представление исходного арифметического выражения в виде бинарного дерева абстрактного синтаксиса (AST). В этом дереве листья являются операндами (числами или переменными), а внутренние узлы — операторами (+, -, *, /, ^).

Метод решения заключается в рекурсивном обходе сформированного дерева снизу вверх (постфиксный обход) с целью поиска узлов разности (-) и их последующей контекстной трансформации.

При обнаружении узла операции вычитания алгоритм анализирует структуру его левого (\$L\$) и правого (\$R\$) поддеревьев для поиска общего сомножителя:

1. Случай «Умножение минус Умножение» (например, $A * B - A * C$ или $B * A - C * A$): Алгоритм попарно сравнивает левые и правые ветви поддеревьев \$L\$ и \$R\$. При обнаружении эквивалентных узлов, данный узел копируется как общий множитель, а оставшиеся части ветвей объединяются новым узлом разности.
2. Случай «Умножение минус Одиночный терм» (например, $A * B - A$): Одиночный терм сравнивается с сомножителями левого поддерева. Если совпадение найдено, одиночный узел извлекается как общий множитель, а на месте вычитаемого создается узел-константа со значением 1.0.
3. Случай «Одиночный терм минус Умножение» (например, $A - A * C$): Логика аналогична предыдущему пункту, единица создается на месте уменьшаемого.

Псевдокод:

ФУНКЦИЯ transform_factor_out(узел)

ЕСЛИ узел == NULL ТОГДА

 ВЕРНУТЬ NULL

КОНЕЦ ЕСЛИ

// Рекурсивный обход поддеревьев (снизу вверх)

```

узел.левый = transform_factor_out(узел.левый)
узел.правый = transform_factor_out(узел.правый)

// Проверка целевой операции
ЕСЛИ узел.оператор == OP_SUB ТОГДА
    L = узел.левый
    R = узел.правый

    общий_множитель = NULL
    остаток_L = NULL
    остаток_R = NULL

// Вариант 1: Оба поддерева — операции умножения
ЕСЛИ L.тип == OPERATOR И L.оператор == OP_MUL И R.тип == OPERATOR И R.оператор ==
OP_MUL ТОГДА
    ЕСЛИ деревья_равны(L.левый, R.левый) ТОГДА
        общий_множитель = L.левый; остаток_L = L.правый; остаток_R = R.правый
    ИНАЧЕ ЕСЛИ деревья_равны(L.левый, R.правый) ТОГДА
        общий_множитель = L.левый; остаток_L = L.правый; остаток_R = R.левый
    ИНАЧЕ ЕСЛИ деревья_равны(L.правый, R.левый) ТОГДА
        общий_множитель = L.правый; остаток_L = L.левый; остаток_R = R.правый
    ИНАЧЕ ЕСЛИ деревья_равны(L.правый, R.правый) ТОГДА
        общий_множитель = L.правый; остаток_L = L.левый; остаток_R = R.левый
    КОНЕЦ ЕСЛИ

// Вариант 2: Левое поддерево — умножение, правое — одиночный элемент
ИНАЧЕ ЕСЛИ L.тип == OPERATOR И L.оператор == OP_MUL ТОГДА
    ЕСЛИ деревья_равны(L.левый, R) ТОГДА
        общий_множитель = L.левый; остаток_L = L.правый; остаток_R = создать_узел_числа(1.0)
    ЕСЛИ деревья_равны(L.правый, R) ТОГДА
        общий_множитель = L.правый; остаток_L = L.левый; остаток_R = создать_узел_числа(1.0)
    КОНЕЦ ЕСЛИ

// Вариант 3: Левое поддерево — одиночный элемент, правое — умножение
ИНАЧЕ ЕСЛИ R.тип == OPERATOR И R.оператор == OP_MUL ТОГДА
    ЕСЛИ деревья_равны(L, R.левый) ТОГДА
        общий_множитель = L; остаток_L = создать_узел_числа(1.0); остаток_R = R.правый
    ЕСЛИ деревья_равны(L, R.правый) ТОГДА
        общий_множитель = L; остаток_L = создать_узел_числа(1.0); остаток_R = R.левый
    КОНЕЦ ЕСЛИ
КОНЕЦ ЕСЛИ

// Если общий множитель успешно выделен, перестраиваем узел дерева
ЕСЛИ общий_множитель != NULL ТОГДА
    новый_узел_разности = создать_узел_оператора(OP_SUB, копия(остаток_L), копия(остаток_R))
    новый_узел_умножения = создать_узел_оператора(OP_MUL, копия(общий_множитель),
новый_узел_разности)
    очистить_дерево(узел)
    ВЕРНУТЬ новый_узел_умножения
КОНЕЦ ЕСЛИ
КОНЕЦ ЕСЛИ

ВЕРНУТЬ узел
КОНЕЦ ФУНКЦИИ

```

7. Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).

План работы:

1. Анализ прототипа: Изучение предоставленной программы преобразования степеней, разбор принципа работы стекового парсера Дейкстры (алгоритм сортировочной станции) и структуры бинарного дерева Node.
2. Проектирование расширения дерева: Разработка алгоритма глубокого сравнения поддеревьев произвольной сложности (age_nodes_equal) для идентификации общих сомножителей (как простых переменных/констант, так и составных выражений в скобках).
3. Реализация бизнес-логики: Написание функции transform_factor_out, покрывающей все три конфигурации расположения общего сомножителя в структуре разности.
4. Коррекция утечек памяти: Модификация процедуры очистки старых поддеревьев при замещении их результирующим узлом умножения.
5. Интеграция и верификация: Обновление функции main, проведение серии тестов.

Соображения по тестированию:

Для подтверждения корректности работы программы необходимо проверить систему на пяти классах эквивалентности входных данных:

- Базовый тест: Простая проверка дистрибутивности на переменных.
- Тест на коммутативность: Проверка того, что порядок сомножителей внутри операции умножения не влияет на обнаружение общего элемента.
- Тест с появлением константной единицы: Проверка корректности создания узла 1 при вынесении операнда целиком.
- Тест на отсутствие изменений: Программа не должна изменять выражения, в которых нет общих элементов или нет операции вычитания.
- Сложносоставной тест: Проверка рекурсивного вынесения элементов во вложенных подвыражениях.

План тестов

- Тест №1 (Базовый):
 - Входное выражение: $a*b - a*c$
 - Ожидаемый результат: $a*(b-c)$
 - Цель: Проверка базового перестроения дерева.
- Тест №2 (Учет коммутативности умножения):
 - Входное выражение: $x*y - z*x$
 - Ожидаемый результат: $x*(y-z)$
 - Цель: Подтверждение того, что алгоритм успешно сопоставляет левые и правые ветви в разных комбинациях ($L.left == R.right$).
- Тест №3 (Вынесение одиночного терма):
 - Входное выражение: $5*x - 5$
 - Ожидаемый результат: $5*(x-1)$
 - Цель: Контроль корректности генерации подменяющего узла NODE_NUMBER со значением 1 для сохранения математической точности.
- Тест №4 (Отсутствие общих сомножителей):
 - Входное выражение: $a*b - c*d$
 - Ожидаемый результат: $a*b - c*d$ (дерево остается неизменным)
 - Цель: Проверка устойчивости алгоритма и отсутствия ложных срабатываний.
- Тест №5 (Вложенная структура):
 - Входное выражение: $(a*b - a*c) * (x*y - x*z)$
 - Ожидаемый результат: $a*(b-c)*x*(y-z)$
 - Цель: Проверка рекурсивного характера обхода дерева (трансформация должна успешно пройти во всех независимых ветвях AST).

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

Листинг окончательного варианта программы (Bash):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_EXPR 256

typedef enum {
    NODE_NUMBER,
    NODE_VARIABLE,
    NODE_OPERATOR
} NodeType;

typedef enum {
    OP_ADD,
```

```

    OP_SUB,
    OP_MUL,
    OP_DIV,
    OP_POW
} Operator;

typedef struct Node {
    NodeType type;
    union {
        double number;
        char variable;
        struct {
            Operator op;
            struct Node* left;
            struct Node* right;
        };
    };
} Node;

typedef struct {
    Node** items;
    int top;
    int capacity;
} Stack;

typedef struct {
    char* items;
    int top;
    int capacity;
} CharStack;

Stack* create_stack(int capacity) {
    Stack* s = (Stack*)malloc(sizeof(Stack));
    s->items = (Node**)malloc(sizeof(Node*) * capacity);
    s->top = -1;
    s->capacity = capacity;
    return s;
}

void push(Stack* s, Node* node) {
    s->items[++s->top] = node;
}

Node* pop(Stack* s) {
    return s->items[s->top--];
}

int is_empty(Stack* s) {
    return s->top == -1;
}

CharStack* create_char_stack(int capacity) {
    CharStack* s = (CharStack*)malloc(sizeof(CharStack));
    s->items = (char*)malloc(sizeof(char) * capacity);
    s->top = -1;
    s->capacity = capacity;
    return s;
}

void push_char(CharStack* s, char c) {
    s->items[++s->top] = c;
}

char pop_char(CharStack* s) {
    return s->items[s->top--];
}

```

```

char peek_char(CharStack* s) {
    return s->items[s->top];
}

int is_empty_char(CharStack* s) {
    return s->top == -1;
}

Node* create_number_node(double value) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->type = NODE_NUMBER;
    node->number = value;
    return node;
}

Node* create_variable_node(char var) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->type = NODE_VARIABLE;
    node->variable = var;
    return node;
}

Node* create_operator_node(Operator op, Node* left, Node* right) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->type = NODE_OPERATOR;
    node->op = op;
    node->left = left;
    node->right = right;
    return node;
}

int get_priority(char op) {
    switch (op) {
        case '+': case '-': return 1;
        case '*': case '/': return 2;
        case '^': return 3;
        default: return 0;
    }
}

int get_priority_from_node(Node* node) {
    if (node->type != NODE_OPERATOR) return 4;
    switch (node->op) {
        case OP_ADD: case OP_SUB: return 1;
        case OP_MUL: case OP_DIV: return 2;
        case OP_POW: return 3;
        default: return 0;
    }
}

void apply_operator(Stack* stack, char op) {
    Node* right = pop(stack);
    Node* left = pop(stack);
    Operator oper;

    switch (op) {
        case '+': oper = OP_ADD; break;
        case '-': oper = OP_SUB; break;
        case '*': oper = OP_MUL; break;
        case '/': oper = OP_DIV; break;
        case '^': oper = OP_POW; break;
        default: return;
    }

    push(stack, create_operator_node(oper, left, right));
}

```

```

Node* parse_expression(const char* expr) {
    Stack* output = create_stack(strlen(expr) * 2);
    CharStack* operators = create_char_stack(strlen(expr) * 2);

    int i = 0;
    while (expr[i] != '\0') {
        if (isspace(expr[i])) {
            i++;
            continue;
        }

        if (isdigit(expr[i]) || expr[i] == '.') {
            char num_str[32];
            int j = 0;
            while (isdigit(expr[i]) || expr[i] == '.') {
                num_str[j++] = expr[i++];
            }
            num_str[j] = '\0';
            push(output, create_number_node(atof(num_str)));
            continue;
        }

        if (isalpha(expr[i])) {
            push(output, create_variable_node(expr[i]));
            i++;
            continue;
        }

        if (expr[i] == '+' || expr[i] == '-' || expr[i] == '*' || expr[i] == '/' || expr[i] == '^') {
            char op = expr[i];
            while (!is_empty_char(operators) && peek_char(operators) != '(') {
                char top_op = peek_char(operators);
                if (get_priority(top_op) >= get_priority(op)) {
                    if (get_priority(top_op) == get_priority(op) && op == '^') {
                        break;
                    }
                    apply_operator(output, pop_char(operators));
                } else {
                    break;
                }
            }
            push_char(operators, op);
            i++;
            continue;
        }

        if (expr[i] == '(') {
            push_char(operators, '(');
            i++;
            continue;
        }

        if (expr[i] == ')') {
            while (!is_empty_char(operators) && peek_char(operators) != '(') {
                apply_operator(output, pop_char(operators));
            }
            if (!is_empty_char(operators) && peek_char(operators) == '(') {
                pop_char(operators);
            }
            i++;
            continue;
        }

        i++;
    }

    while (!is_empty_char(operators)) {

```

```

    apply_operator(output, pop_char(operators));
}

Node* result = pop(output);

free(output->items);
free(output);
free(operators->items);
free(operators);

return result;
}

Node* copy_node(Node* node) {
    if (node == NULL) return NULL;

    if (node->type == NODE_NUMBER) {
        return create_number_node(node->number);
    } else if (node->type == NODE_VARIABLE) {
        return create_variable_node(node->variable);
    } else {
        return create_operator_node(node->op,
                                     copy_node(node->left),
                                     copy_node(node->right));
    }
}

int are_nodes_equal(Node* n1, Node* n2) {
    if (n1 == NULL && n2 == NULL) return 1;
    if (n1 == NULL || n2 == NULL) return 0;
    if (n1->type != n2->type) return 0;

    if (n1->type == NODE_NUMBER) {
        return n1->number == n2->number;
    } else if (n1->type == NODE_VARIABLE) {
        return n1->variable == n2->variable;
    } else {
        if (n1->op != n2->op) return 0;
        return are_nodes_equal(n1->left, n2->left) && are_nodes_equal(n1->right, n2->right);
    }
}

Node* transform_factor_out(Node* node) {
    if (node == NULL) return NULL;

    if (node->type == NODE_OPERATOR) {
        node->left = transform_factor_out(node->left);
        node->right = transform_factor_out(node->right);

        if (node->op == OP_SUB) {
            Node* L = node->left;
            Node* R = node->right;

            Node* common = NULL;
            Node* rem_L = NULL;
            Node* rem_R = NULL;

            if (L->type == NODE_OPERATOR && L->op == OP_MUL &&
                R->type == NODE_OPERATOR && R->op == OP_MUL) {

                if (are_nodes_equal(L->left, R->left)) {
                    common = L->left; rem_L = L->right; rem_R = R->right;
                } else if (are_nodes_equal(L->left, R->right)) {
                    common = L->left; rem_L = L->right; rem_R = R->left;
                } else if (are_nodes_equal(L->right, R->left)) {
                    common = L->right; rem_L = L->left; rem_R = R->right;
                } else if (are_nodes_equal(L->right, R->right)) {

```

```

        common = L->right; rem_L = L->left; rem_R = R->left;
    }
}
else if (L->type == NODE_OPERATOR && L->op == OP_MUL) {
    if (are_nodes_equal(L->left, R)) {
        common = L->left; rem_L = L->right; rem_R = create_number_node(1.0);
    } else if (are_nodes_equal(L->right, R)) {
        common = L->right; rem_L = L->left; rem_R = create_number_node(1.0);
    }
}
else if (R->type == NODE_OPERATOR && R->op == OP_MUL) {
    if (are_nodes_equal(L, R->left)) {
        common = L; rem_L = create_number_node(1.0); rem_R = R->right;
    } else if (are_nodes_equal(L, R->right)) {
        common = L; rem_L = create_number_node(1.0); rem_R = R->left;
    }
}

if (common != NULL) {
    Node* final_common = copy_node(common);
    Node* final_rem_L = (rem_L->type == NODE_NUMBER && rem_L->number == 1.0 &&
rem_L != L && rem_L != R) ? rem_L : copy_node(rem_L);
    Node* final_rem_R = (rem_R->type == NODE_NUMBER && rem_R->number == 1.0 &&
rem_R != L && rem_R != R) ? rem_R : copy_node(rem_R);

    Node* sub_node = create_operator_node(OP_SUB, final_rem_L, final_rem_R);
    Node* result = create_operator_node(OP_MUL, final_common, sub_node);

    free_tree(node);
    return result;
}
}

return node;
}

void print_tree(Node* node, int depth, const char* prefix) {
    if (node == NULL) return;

    for (int i = 0; i < depth; i++) {
        printf(" ");
    }

    switch (node->type) {
        case NODE_NUMBER:
            printf("%s%.2f\n", prefix, node->number);
            break;
        case NODE_VARIABLE:
            printf("%s%c\n", prefix, node->variable);
            break;
        case NODE_OPERATOR:
            {
                char op_char;
                switch (node->op) {
                    case OP_ADD: op_char = '+'; break;
                    case OP_SUB: op_char = '-'; break;
                    case OP_MUL: op_char = '*'; break;
                    case OP_DIV: op_char = '/'; break;
                    case OP_POW: op_char = '^'; break;
                }
                printf("%s%c\n", prefix, op_char);
            }
        print_tree(node->left, depth + 1, "L:");
        print_tree(node->right, depth + 1, "R:");
        break;
    }
}

```



```

}

void tree_to_string(Node* node, char* buffer, int* pos) {
    if (node == NULL) return;

    if (node->type == NODE_NUMBER) {
        char num[32];
        if (node->number == (int)node->number) {
            sprintf(num, "%d", (int)node->number);
        } else {
            sprintf(num, "%.2f", node->number);
            char* dot = strchr(num, '.');
            if (dot) {
                char* end = num + strlen(num) - 1;
                while (end > dot && *end == '0') end--;
                if (end == dot) end--;
                *(end + 1) = '\0';
            }
        }
        strcpy(buffer + *pos, num);
        *pos += strlen(num);
    } else if (node->type == NODE_VARIABLE) {
        buffer[(*pos)++] = node->variable;
    } else {
        int need_paren_left = 0, need_paren_right = 0;

        if (node->left->type == NODE_OPERATOR &&
            get_priority_from_node(node->left) < get_priority_from_node(node)) {
            need_paren_left = 1;
        }

        if (node->right->type == NODE_OPERATOR) {
            int right_priority = get_priority_from_node(node->right);
            int op_priority = get_priority_from_node(node);
            if (right_priority < op_priority ||
                (right_priority == op_priority && (node->op == OP_SUB || node->op == OP_DIV))) {
                need_paren_right = 1;
            }
        }

        if (need_paren_left) buffer[(*pos)++] = '(';
        tree_to_string(node->left, buffer, pos);
        if (need_paren_left) buffer[(*pos)++] = ')';

        char op_char;
        switch (node->op) {
            case OP_ADD: op_char = '+'; break;
            case OP_SUB: op_char = '-'; break;
            case OP_MUL: op_char = '*'; break;
            case OP_DIV: op_char = '/'; break;
            case OP_POW: op_char = '^'; break;
        }
        buffer[(*pos)++] = op_char;

        if (need_paren_right) buffer[(*pos)++] = '(';
        tree_to_string(node->right, buffer, pos);
        if (need_paren_right) buffer[(*pos)++] = ')';
    }
}

void free_tree(Node* node) {
    if (node == NULL) return;
    if (node->type == NODE_OPERATOR) {
        free_tree(node->left);
        free_tree(node->right);
    }
    free(node);
}

```

```

}

int main() {
    char expr[MAX_EXPR];

    printf("Enter expression: ");
    fflush(stdout);

    if (fgets(expr, MAX_EXPR, stdin) == NULL) {
        printf("Error reading input\n");
        return 1;
    }

    expr[strcspn(expr, "\n")] = '\0';

    if (strlen(expr) == 0) {
        printf("Empty expression\n");
        return 1;
    }

    printf("\nOriginal expression: %s\n", expr);

    Node* root = parse_expression(expr);
    if (root == NULL) {
        printf("Parse error\n");
        return 1;
    }

    printf("\nTree BEFORE transformation:\n");
    print_tree(root, 0, "");

    Node* transformed = transform_factor_out(root);

    printf("\nTree AFTER transformation:\n");
    print_tree(transformed, 0, "");

    char result_str[MAX_EXPR * 2] = {0};
    int pos = 0;
    tree_to_string(transformed, result_str, &pos);
    printf("\nResult: %s\n", result_str);

    free_tree(transformed);

    return 0;
}

```

Распечатка протокола:

1. Тест №1.

Enter expression: a*b - a*c

Original expression: a*b - a*c

Tree BEFORE transformation:

-

L:*

L:a

R:b

R:*

L:a

R:c

Tree AFTER transformation:

*

L:a

R:-

L:b

R:c

Result: $a*(b-c)$

2. Тест №2.

Enter expression: $x*y - z*x$

Original expression: $x*y - z*x$

Tree BEFORE transformation:

-

L:*

L:x

R:y

R:*

L:z

R:x

Tree AFTER transformation:

*

L:x

R:-

L:y

R:z

Result: $x*(y-z)$

3. Тест №3.

Enter expression: $5*x - 5$

Original expression: $5*x - 5$

Tree BEFORE transformation:

-

L:*

L:5.00

R:x

R:5.00

Tree AFTER transformation:

*

L:5.00

R:-

L:x

R:1.00

Result: $5*(x-1)$

4. Тест №4.

Enter expression: $a*b - c*d$

Original expression: $a*b - c*d$

Tree BEFORE transformation:

-

L:*

L:a

R:b

R:*

L:c

R:d

Tree AFTER transformation:

-

L:*

L:a

R:b

R:*

L:c

R:d

Result: $a*b-c*d$

5. Тест №5.

Enter expression: $(a*b - a*c) * (x*y - x*z)$

Original expression: $(a*b - a*c) * (x*y - x*z)$

Tree BEFORE transformation:

*

L:-

L:*

L:a

R:b

R:*

L:a

R:c

R:-

L:*

L:x

R:y

R:*

L:x

R:z

Tree AFTER transformation:

*

L:*

L:a

R:-

L:b

R:c

R:*

L:x

R:-

L:y

R:z

Result: $a*(b-c)*x*(y-z)$

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе лабораторной работы разработаны два скрипта (Bash и Python) для создания копий файла с автоматической генерацией имён путём добавления последовательных букв или цифр. Реализована корректная обработка переноса разрядов в буквенном режиме (после 'z' идёт 'za'). Скрипты включают проверку входных данных и обработку ошибок. Работа позволила закрепить навыки файлового ввода-вывода, обработки аргументов командной строки и строковых операций. Оба скрипта успешно протестированы в Ubuntu.

Подпись студента _____